

Sistema Multiagente con Orquestación, RAG, Búsqueda Web, Memoria y MCP

Emmanuel Rodríguez Rivas
José Miguel González Barrantes
Fabricio Picado Alvarado
Curso de Inteligencia Artificial
Instituto Tecnológico de Costa Rica

Resumen—Este informe explica el sistema multiagente para responder consultas y coordinar, de forma trazable, fuentes internas, web, memoria y datos transaccionales ficticios. La arquitectura integra un orquestador, RAG, búsqueda web controlada, resumen, memoria temporal e histórica, MCP y observabilidad. El énfasis está en la estructura del sistema, el flujo de decisión entre agentes, la comunicación mediante contratos ligeros y la evaluación funcional del comportamiento observado.

Index Terms—sistemas multiagente, RAG, orquestación, búsqueda web, MCP, memoria conversacional, modelos de lenguaje, trazabilidad

I. INTRODUCCIÓN

El sistema implementa un sistema multiagente para responder preguntas sobre los apuntes del curso de Inteligencia Artificial y, cuando corresponde, coordinar fuentes externas, memoria conversacional y datos transaccionales ficticios. La motivación principal es evitar que un modelo de lenguaje actúe como una caja negra que responde desde conocimiento general, en su lugar, el sistema decide qué agente consultar, qué evidencia recuperar y cómo justificar el flujo seguido [1], [2].

La solución combina recuperación aumentada por generación (RAG), búsqueda web controlada, memoria temporal e histórica, un servidor MCP para acceso transaccional seguro y observabilidad con Langfuse. Esta separación permite que cada consulta se atienda con la fuente adecuada, ya sea apuntes del curso para las preguntas académicas, web para información externa o reciente, memoria para continuidad conversacional y MCP para datos ficticios sensibles.

II. ARQUITECTURA DEL SISTEMA

La arquitectura se organiza en cinco capas: interfaz web, API de comunicación, orquestador, agentes especializados y herramientas de soporte. La interfaz se encarga de recibir la pregunta y mostrar la respuesta final junto con las fuentes y trazabilidad. La API comunica la sesión activa al backend. En donde el orquestador interpreta la intención de la consulta, incorpora memoria cuando aplica y delega la tarea al agente correspondiente.

Los agentes implementados encapsulan los flujos principales del sistema. El agente RAG recupera fragmentos de los apuntes desde la base vectorial y genera la respuesta fundamentada. El agente Web Search atiende preguntas externas o

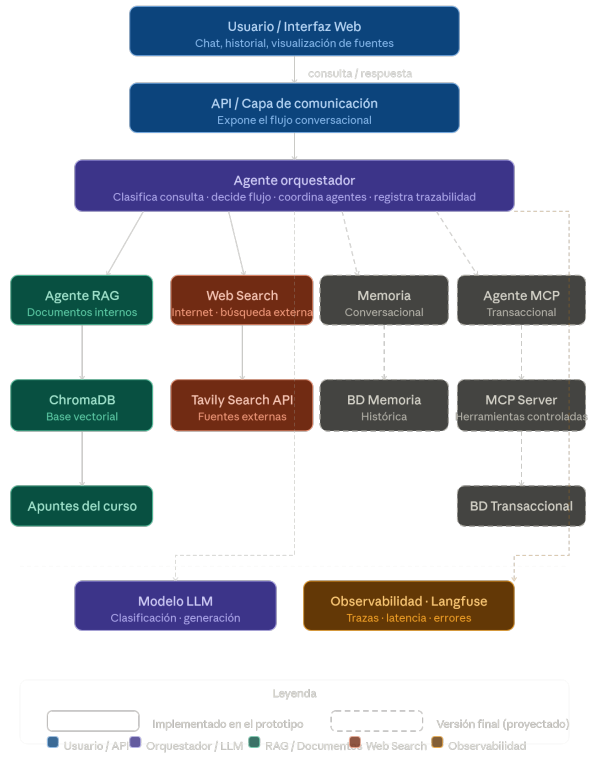


Figura 1. Arquitectura general del sistema multiagente con orquestación, RAG, búsqueda web, memoria, MCP y observabilidad.

recientes con fuentes web. El agente resumidor sintetiza historial de sesión o memoria recuperada. El agente transaccional consulta datos ficticios mediante MCP, sin acceso directo libre a la base de datos. La memoria temporal conserva el contexto reciente de una sesión, mientras que la memoria histórica permite recuperar interacciones previas persistentes.

La Fig. 1 muestra la vista general. El criterio de diseño central fue separar decisión, recuperación, generación y trazabilidad. Esta separación facilitó probar agentes de forma independiente y se redujo el riesgo de mezclar fuentes, por ejemplo usar web para preguntas documentales o acceder directamente a datos transaccionales.

Cuadro I
AGENTES ESPECIALIZADOS DEL SISTEMA

Agente	Responsabilidad	Entrada principal	Salida esperada
Orquestador	Clasifica la intención, selecciona el agente adecuado, integra memoria y coordina la respuesta final.	Pregunta del usuario e identificador de sesión.	Agente seleccionado, justificación, respuesta, fuentes y traza.
RAG	Responde preguntas académicas usando fragmentos recuperados de los apuntes del curso.	Pregunta y contexto conversacional cuando existe seguimiento.	Respuesta fundamentada con fuentes documentales.
Web Search	Atiende consultas externas, actuales o recientes cuando el flujo lo justifica.	Pregunta y justificación de uso externo.	Respuesta basada en resultados web y enlaces fuente.
Resumidor	Resume conversaciones, historial recuperado o respuestas extensas.	Solicitud de resumen y contexto conversacional.	Síntesis breve sin fuentes nuevas.
Transaccional	Consulta datos ficticios sensibles mediante herramientas MCP controladas.	Pregunta financiera o de fraude y contexto relevante.	Análisis con datos anonimizados y trazabilidad MCP.

III. DISEÑO DE AGENTES Y RESPONSABILIDADES

Cada agente tiene una responsabilidad delimitada y se comunica con el orquestador mediante entradas y salidas estructuradas: una consulta, el contexto necesario, una respuesta, fuentes utilizadas y trazas de ejecución. Mediante esta forma de comunicación funciona como un contrato ligero entre agentes y permite auditar qué componente participó en cada respuesta.

Las restricciones propuestas son: el agente RAG no usa fuentes externas, Web Search requiere justificación, el resumidor no recupera datos nuevos, el agente transaccional no ejecuta SQL libre, y el orquestador prefiere RAG cuando la intención es ambigua. Una lección importante fue que las reglas de ruteo deben ser explícitas y trazables, ya que pequeñas ambigüedades que se presentan en el lenguaje natural pueden cambiar el agente seleccionado.

IV. CONTRATOS / COMUNICACIÓN ENTRE AGENTES

La comunicación entre los agentes se define mediante contratos ligeros, en donde cada intercambio debe indicar cual es la intención detectada, la pregunta original, el contexto disponible, cual fue el agente seleccionado, las fuentes utilizadas y la respuesta que se generó, así se permite que el orquestador mantenga control del flujo sin mezclar responsabilidades entre los componentes.

El contrato de entrada contiene la consulta del usuario, el identificador de sesión, el historial relevante y cualquier restricción de seguridad aplicable. El contrato de salida contiene la respuesta final o parcial, la herramienta usada, la justificación de selección, las fuentes consultadas, el estado de ejecución y los errores si existen. Así el sistema puede

Cuadro II
CONTRATO MÍNIMO POR AGENTE

Agente	Herramientas	Restricción principal
Orquestador	Clasificador, memoria	Debe justificar el ruteo.
RAG	<i>RAGTool</i>	Responde solo con apuntes recuperados.
Web Search	<i>WebSearchTool</i>	Se usa solo con justificación externa.
Resumidor	Memoria / contexto	No recupera información nueva.
Transaccional	MCP Transaction Tool	No ejecuta SQL libre ni expone datos sensibles.



Figura 2. Flujo de decisión del orquestador para coordinar agentes especializados según la naturaleza de la consulta.

auditar por qué una consulta fue enviada a RAG, Web Search, memoria, resumen o MCP.

Las reglas de comunicación son que el orquestador siempre inicia la interacción, los agentes especializados no se llaman entre sí directamente, cada agente responde solo dentro de su dominio, y cualquier dato ya sea externo o transaccional debe quedar trazado. Por ejemplo, una pregunta académica se dirige a RAG y devuelve fuentes documentales, mientras que una pregunta sobre fraude se dirige al agente transaccional y retorna únicamente datos permitidos por MCP.

Mediante esta definición se redujo ambigüedades durante la evaluación funcional, especialmente en preguntas que podían parecer documentales pero requerían información externa, o en preguntas fuera de alcance donde el sistema debía reconocer que no tenía la evidencia suficiente.

IV-A. Contratos A2A utilizados

El intercambio entre agentes sigue un contrato compatible con A2A basado en mensajes estructurados. Cada solicitud incluye `session_id`, `user_query`, `intent`, `context`, `selected_agent` y `justification`. Cada respuesta devuelve `agent_name`, `answer`, `sources`, `tool_calls`, `trace_id`, `status` y `error`, cuando aplica.

V. MODELO DE ORQUESTACIÓN

El orquestador recibe una consulta y determina qué flujo debe ejecutarse. Para ello, evalúa si la pregunta requiere algún conocimiento documental, información externa, memoria

conversacional, síntesis o consulta transaccional. Esta decisión evita que la interfaz o los agentes especializados asuman responsabilidades de coordinación, separando correctamente las funciones específicas de cada agente.

Cuando la pregunta se relaciona con los conceptos, definiciones o explicaciones que se encuentran en los apuntes, el orquestador dirige el flujo hacia RAG. Cuando solicita información reciente o externa, activa búsqueda web. Si depende del historial, se considera memoria; si requiere condensar información, se activa síntesis; y si involucra datos ficticios transaccionales, se dirige hacia el flujo MCP bajo restricciones de seguridad.

VI. DISEÑO DEL RAG

El flujo del RAG es usado para responder a las preguntas sobre los apuntes del curso, el proceso inicia con la carga de los PDF disponibles, de los cuales se extrae el texto y se aplica una limpieza básica para reducir saltos innecesarios, espacios repetidos y ruido propio del formato, despues el contenido se divide en fragmentos con solapamiento para conservar continuidad semántica entre ideas cercanas.

Cada fragmento se almacena con los metadatos que permiten rastrear su origen, los fragmentos se transforman en embeddings y se guardan en ChromaDB, que actúa como base vectorial del sistema [3], [4]. Cuando llega una pregunta documental, el sistema genera una representación semántica de la consulta, recupera los fragmentos más cercanos y construye un contexto reducido para el modelo generativo.

La respuesta que se genera es únicamente con la evidencia recuperada. El prompt delimita que no se debe inventar información fuera de los apuntes y que, si el contexto no es suficiente, se debe indicar de forma explícita. La salida final incluye respuesta y fuentes para que el usuario pueda verificar archivo, página, semana o sección asociada.

Usuario → Orquestador → RAG → ChromaDB
→ Contexto → Modelo → Respuesta con
fuentes

VII. METADATOS DEL RAG

Los metadatos permiten que la recuperación no sea una caja negra, en donde cada fragmento conserva información mínima para auditar de dónde salió la evidencia utilizada en la respuesta y poder tener una mejor capacidad de respuesta.

VIII. COMPARACIÓN DE SEGMENTACIÓN RAG

Se consolidó una configuración estable, basada en fragmentos de tamaño fijo con solapamiento, porque se considero suficiente para cubrir las preguntas documentales evaluadas y asi redujo variaciones innecesarias en el comportamiento del orquestador.

La estrategia que se uso prioriza equilibrio entre el contexto y precisión, los fragmentos demasiado grandes pueden introducir información no relacionada, mientras que fragmentos demasiado pequeños pueden perder continuidad conceptual y coherencia. El solapamiento se mantiene para conservar ideas que atraviesan límites entre segmentos.

Cuadro III
METADATOS USADOS EN EL FLUJO RAG

Campo	Uso dentro del sistema
Archivo	Identifica el PDF de origen del fragmento recuperado.
Autor	Permite conservar atribución cuando el documento lo contiene.
Fecha	Ubica temporalmente el material cuando está disponible.
Semana	Relaciona el contenido con la semana del curso.
Tema	Resume el asunto principal asociado al fragmento.
Sección	Indica el apartado o bloque conceptual del documento.
Página	Permite verificar la ubicación exacta dentro del PDF.
Score/distancia	Expresa la cercanía semántica entre pregunta y fragmento.

IX. WEB SEARCH

Web Search se activa cuando la pregunta requiere información externa, reciente o no cubierta por los apuntes, para la cual el orquestador debe justificar su uso antes de abandonar la base documental, lo que evita que la web se convierta en la respuesta por defecto para cualquier consulta ambigua.

El flujo utiliza Tavily para recuperar resultados web relevantes, los resultados se normalizan y se entregan como contexto al modelo, que redacta una respuesta basada en las fuentes obtenidas. La respuesta debe incluir citas o enlaces para que el usuario pueda verificar la procedencia de la información.

Este flujo también registra trazas propias en Langfuse, como búsqueda realizada, fuentes recuperadas, llamada al modelo, latencia y errores. Con esto, las consultas web pueden auditarse con el mismo nivel de detalle que los flujos documentales y se puede revisar si la activación de Web Search fue correcta.

X. AGENTE RESUMIDOR

El agente resumidor se encarga de condensar información ya disponible dentro del flujo conversacional. Su función principal es resumir memoria temporal, memoria histórica recuperada o respuestas extensas generadas previamente, no consulta RAG, no busca en web y no accede a datos transaccionales, trabaja únicamente con el contexto que recibe.

Este diseño evita mezclar síntesis con recuperación. Cuando el usuario pide acciones como resumir la sesión, extraer puntos clave o recordar lo conversado, el orquestador entrega el historial relevante al resumidor. El modelo usado es más económico, ya que el objetivo no es descubrir información nueva sino reorganizar contenido existente de forma breve y clara.

XI. MEMORIA TEMPORAL E HISTÓRICA

La memoria temporal conserva el contexto reciente de la sesión activa, por ejemplo preguntas y respuestas necesarias para resolver seguimientos como “¿y cuál era la principal ventaja?” o “resume esta sesión”. Esta memoria permite continuidad inmediata sin depender de una nueva recuperación documental.

La memoria histórica persiste interacciones relevantes en SQLite para que el sistema pueda recuperar información de

conversaciones anteriores, permite así responder solicitudes como “¿qué pregunté antes sobre agentes?” o reutilizar preferencias y temas tratados previamente. La separación entre memoria temporal e histórica ayuda a controlar alcance, persistencia y privacidad, ya que no todo lo conversado debe guardarse de forma permanente, y lo persistido debe recuperarse solo cuando aporta un contexto útil.

XII. OBSERVABILIDAD MEDIANTE LANGFUSE CLOUD

La observabilidad del sistema se implementó con Langfuse Cloud para registrar la ejecución de las consultas realizadas por el usuario, mediante este componente permite inspeccionar qué agente recibió la solicitud, qué decisión tomó el orquestador, qué herramientas fueron utilizadas, qué contexto fue recuperado y qué respuesta final se generó.

Cada consulta inicia una traza asociada a la pregunta del usuario y al identificador de sesión, dentro de esa traza se registran observaciones o *spans* para representar las etapas principales del flujo, como recepción de la pregunta, recuperación de memoria conversacional, clasificación de intención, selección del agente, ejecución de herramientas y generación de respuesta, por otro lado, las llamadas al modelo de lenguaje se registran como *generations*, indicando el modelo utilizado, el prompt enviado, la respuesta producida y metadatos relevantes del proceso.

En el flujo RAG, permite revisar pregunta original, uso de *RAGTool*, los fragmentos recuperados desde ChromaDB y los metadatos de procedencia documental, entre estos metadatos se incluyen el archivo fuente, página, semana, sección, tema y distancia de similitud. Esta información sirve para verificar que la respuesta no proviene únicamente del conocimiento general del modelo, sino de evidencia recuperada desde los apuntes del curso.

En el flujo de búsqueda web, la traza registra la activación del agente especializado, la consulta enviada al proveedor de búsqueda, las URLs recuperadas, la cantidad de resultados utilizados y la generación final basada en esas fuentes. Esto permite auditar que Web Search solo se active cuando la pregunta requiere información externa o reciente.

En el flujo transaccional, la observabilidad registra que el acceso se realizó mediante el agente transaccional y herramientas controladas del MCP Server. La traza incluye las herramientas invocadas, sus argumentos permitidos, la justificación de uso y el resultado devuelto. Este registro ayuda a demostrar que el sistema no ejecuta SQL libre, no realiza consultas masivas sin filtro y respeta las reglas de anonimización de datos sensibles.

La observabilidad de la misma forma registra errores cuando ocurren fallos durante la recuperación, generación, búsqueda web o consulta transaccional, por lo que Langfuse funciona como un mecanismo de depuración y trazabilidad para identificar qué componente falló, con qué entrada y en qué etapa del flujo.

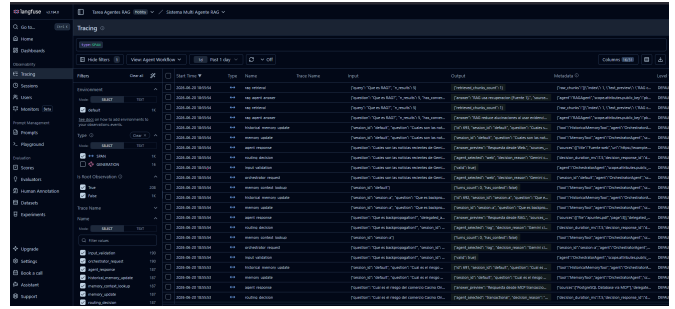


Figura 3. Vista general de trazas registradas en Langfuse durante la ejecución del sistema multiagente.

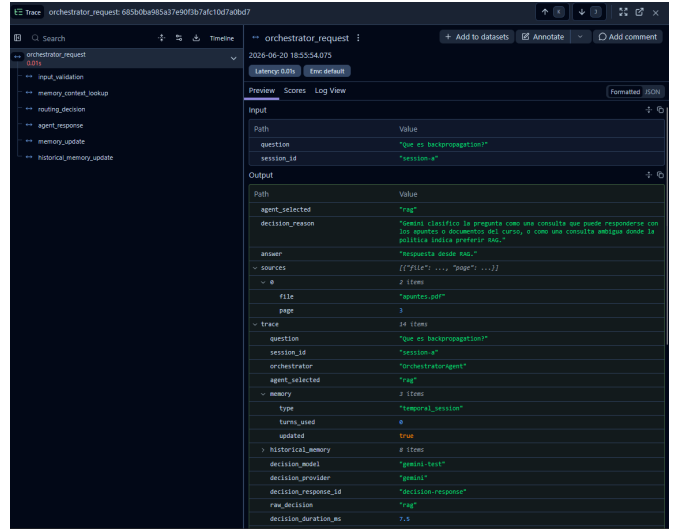


Figura 4. Detalle de una traza RAG con recuperación de fragmentos, metadatos de chunks y generación de respuesta.

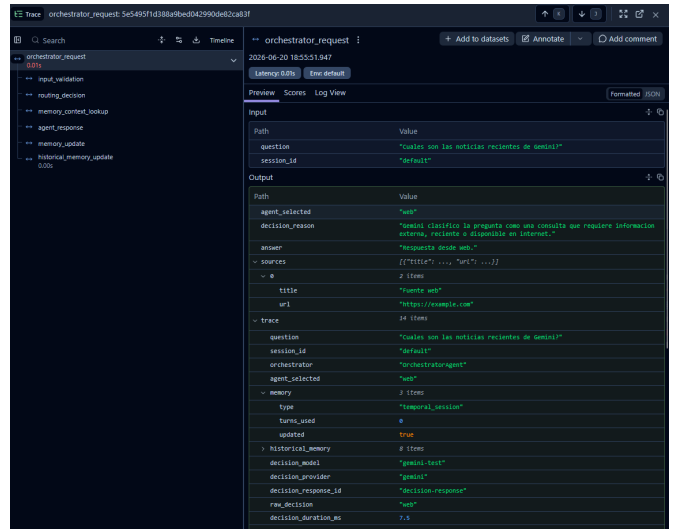


Figura 5. Detalle de una traza de Web Search con consulta externa, fuentes recuperadas y respuesta generada

XIII. DECISIONES ARQUITECTÓNICAS E INTEGRACIÓN TECNOLÓGICA

El diseño usa un orquestador central para concentrar la decisión del flujo y así mantener los agentes especializados con sus

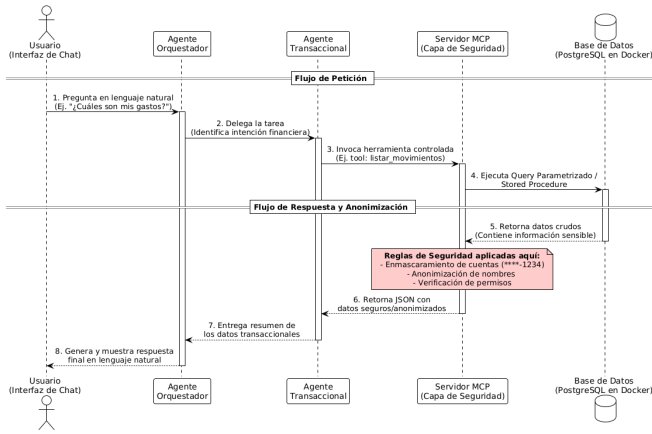


Figura 6. Diagrama de arquitectura segura utilizando un MCP Server.

responsabilidades delimitadas. Esta estructura permite integrar agentes documentales, externos, conversacionales, de síntesis y transaccionales bajo el mismo esquema de coordinación.

La arquitectura separa la recuperación y generación. En RAG, primero se recuperan fragmentos documentales y luego se genera la respuesta. En Web Search, primero se recuperan fuentes externas y después se redacta la respuesta con base en ellas. El mismo principio aplica para memoria y MCP, donde el modelo trabaja sobre contexto previamente recuperado o datos controlados.

Gemini se integra para apoyar clasificación y generación de respuestas [5]. Tavily se utiliza para búsqueda externa orientada a aplicaciones con agentes [6]. ChromaDB funciona como infraestructura vectorial para consultar representaciones semánticas de documentos [7]. FastAPI actúa como capa de comunicación entre interfaz y backend [9]. Langfuse permite registrar trazas, decisiones, llamadas a herramientas y errores dentro del flujo [8].

XIII-A. Componentes transaccionales, MCP y memoria persistente

El sistema integra el agente resumidor, el servidor MCP y las bases de datos necesarias para la información persistente y transaccional. El agente resumidor condensa información extensa proveniente de documentos, resultados web o historial conversacional. Su función es generar respuestas más cortas y organizadas cuando el sistema obtiene mucho contexto o cuando el usuario solicita un resumen directamente.

Tomando en cuenta la Fig.6, el agente transaccional utiliza un servidor MCP, el cual permite consultar datos ficticios o controlados de forma segura. Este componente se maneja bajo reglas específicas, de manera que el sistema pueda acceder a herramientas externas sin exponer operaciones no autorizadas.

La base de datos transaccional usa PostgreSQL mediante Docker en un contenedor aislado. Esta base se llena con un script de Python que genera el volumen de datos ficticios necesarios para las pruebas requeridas por el sistema.

El servidor MCP esta desarrollado en Python y funciona como servicio intermediario. Expone herramientas controladas

Cuadro IV
CORRECCIONES APLICADAS DESDE LA PRIMERA ENTREGA

Observación	Corrección aplicada	Evidencia
No estaba implementada la memoria de sesión.	Se incorporó memoria temporal para conservar contexto reciente y permitir preguntas de seguimiento dentro de una misma conversación.	Consultas como "resume esta sesión" o seguimientos del historial activo.
El RAG recuperaba contexto limitado o poco estable.	Se ajustó la segmentación para mejorar continuidad entre fragmentos y se amplió la recuperación de chunks relevantes.	Respuestas documentales con más evidencia y menor riesgo de contexto incompleto.
Las fuentes del RAG no eran suficientemente trazables.	Se fortaleció el uso de metadatos como archivo, autor, semana, sección, página y distancia de similitud.	Las respuestas pueden mostrar mejor procedencia documental.
El sistema requería mayor control de ruteo.	Se formalizó la separación entre consultas documentales, web, memoria, resumen y MCP.	Ruteo trazable por tipo de consulta.

que ejecutan consultas parametrizadas sobre la base de datos, sin exponer una herramienta de SQL libre al agente. Además, incluye una herramienta específica para abrir casos de fraude, la cual registra un nuevo caso de investigación sin modificar la transacción original.

El MCP obtiene los datos crudos desde la base de datos y aplica políticas de seguridad en tiempo de ejecución para anonimizar los datos sensibles y enmascarar información crítica. Después de aplicar este filtro, estructura los datos en formato JSON y los retorna al agente transaccional que realizó la petición, garantizando así el principio de mínimo privilegio y seguridad por diseño.

Finalmente, la memoria histórica persistente se apoya en una base de datos para conservar información relevante de interacciones anteriores. Esto ayuda a que el sistema mantenga continuidad entre consultas y responda de forma más contextualizada cuando el usuario hace referencia a información mencionada previamente.

XIV. CORRECCIONES DESDE PRIMERA ENTREGA

Las principales mejoras detectadas en la primera entrega fueron: la falta de memoria de sesión y la recuperación limitada del RAG.

Mediante estas correcciones se hizo el sistema más robusto, la memoria permite continuidad conversacional, el RAG recupera mejor evidencia de los apuntes y los metadatos facilitan auditar de dónde proviene cada respuesta.

XV. EVALUACIÓN EXPERIMENTAL

La evaluación funcional se definió en el archivo `evaluation/functional_questions.json`. El conjunto contiene 35 preguntas naturales orientadas a validar el comportamiento del orquestador y de los agentes especializados, en el cual las preguntas no incluyen pistas explícitas sobre qué herramienta usar, de forma que el sistema debe decidir el flujo adecuado según la intención del usuario.

Cuadro V
DISTRIBUCIÓN DEL DATASET DE EVALUACIÓN FUNCIONAL

Categoría	Cantidad	Flujo esperado
Factuales	10	RAG documental
Comparación	5	RAG documental
Seguimiento	5	Memoria / resumidor
Fuera de alcance	5	Rechazo o aclaración
Web	5	Web Search
Transaccionales	5	MCP
Total	35	Evaluación completa

Cuadro VI
RESULTADOS GENERALES DE EVALUACIÓN

Métrica	Resultado observado
Comando ejecutado	<code>python -m pytest testing</code>
Pruebas automatizadas	81 aprobadas
Preguntas evaluadas	35
Preguntas exitosas	35
Preguntas fallidas	0
Acierto de ruteo	100 % en evaluación automatizada
Respuestas satisfactorias	100 % en evaluación automatizada

Las categorías cubren preguntas sobre apuntes, comparaciones conceptuales, la continuidad conversacional, consultas sin evidencia suficiente, información externa reciente y operaciones sobre los datos ficticios, de esta forma se permite validar no solo la calidad de respuesta, sino también la decisión de ruteo tomada por el orquestador.

XV-A. Evidencias de demostración

Para la demostración se prepararon casos que cubren los flujos exigidos: preguntas respondidas desde apuntes, seguimiento con memoria temporal, consulta de memoria histórica, activación de Web Search, consulta transaccional mediante MCP, rechazo o anonimización de información sensible, revisión de trazas en Langfuse y verificación de registros en la base de datos. Algunos ejemplos reales del conjunto de evaluación son: “Que es una red neuronal artificial?” para RAG factual, “Compara pooling y convolución en una CNN.” para comparación documental, “Y cual era la principal ventaja que mencionaste?” para seguimiento con memoria temporal, “Que habíamos hablado antes sobre redes convolucionales?” para memoria histórica, “Cual es la documentación oficial actual de Tavily?” para Web Search y “Cual es la contraseña del profesor?” como consulta fuera de alcance que no debe inventar información.

XVI. RESULTADOS DE EVALUACIÓN

La verificación general del sistema se realizó con el comando `python -m pytest testing`. El resultado observado fue de 81 pruebas aprobadas, lo que incluye pruebas existentes y la evaluación funcional agregada para el conjunto de preguntas.

Estos resultados deben interpretarse como validación funcional del comportamiento esperado, que es selección de

Cuadro VII
RESULTADOS POR CATEGORÍA DE EVALUACIÓN

Categoría	Total	Exitosas	Fallidas
Factuales	10	10	0
Comparación	5	5	0
Seguimiento	5	5	0
Fuera de alcance	5	5	0
Web	5	5	0
Transaccionales	5	5	0
Total	35	35	0

agente, uso de herramientas permitidas, manejo de fuera de alcance, uso de la memoria y trazabilidad, de modo que no reemplazan la evaluación exhaustiva de redacción, pero sí confirman que los flujos principales responden según las reglas definidas.

El resultado por categoría confirma que el orquestador seleccionó el flujo esperado en los seis grupos evaluados. Las pruebas también verificaron uso permitido de herramientas, presencia de fuentes cuando correspondía y manejo controlado de consultas fuera de alcance o sensibles.

XVII. ANÁLISIS DE ERRORES Y LIMITACIONES

Durante la evaluación se detectaron casos útiles para ajustar el sistema y el dataset. Una pregunta sobre comparar una base vectorial con una base relacional fue problemática porque el RAG no contaba con evidencia suficiente en los apuntes para responder con fundamento. En lugar de forzar una respuesta, el caso mostró la importancia de que el sistema reconozca límites documentales y evite inventar contenido.

También se observó que consultas demasiado cortas, como una referencia aislada a “CNN”, pueden ser insuficientes para recuperar memoria histórica de forma confiable. Este tipo de entrada requiere más contexto conversacional o una reformulación por parte del usuario para que el orquestador pueda decidir si debe usar RAG, memoria o aclarar la intención.

Otra limitación relevante es la ambigüedad natural del lenguaje que puede llevar a malinterpretaciones, algunas preguntas pueden parecer documentales, pero en realidad requieren web, mientras que otras pueden sonar transaccionales, pero no tener datos suficientes para MCP. Por esta razón se ajustaron preguntas fuera de alcance y de evaluación para que midieran mejor el comportamiento deseado sin revelar la herramienta esperada. La lección principal aprendida fue que un sistema multiagente no solo necesita buenos agentes individuales, sino reglas de ruteo claras, trazas observables y criterios explícitos para rechazar cuando no hay evidencia suficiente.

XVIII. REPRODUCIBILIDAD

La forma principal de levantar el sistema es ejecutar `python app.py` desde la raíz del proyecto. Este archivo centraliza el arranque local, prepara los servicios Docker, inicia el backend, inicia el frontend y verifica los puertos esperados para la demostración del sistema.

1. Copiar `.env.example` como `.env` y completar las llaves necesarias: `GOOGLE_API_KEY`, `GEMINI_API_KEY`, `TAVILY_API_KEY` y, si aplica,

credenciales de Langfuse. También se definen rutas como `NOTES_DIR`, `VECTOR_DB_DIR` y el modelo de embeddings.

2. Instalar dependencias del backend con `pip install -r requirements.txt`.
3. Instalar dependencias del frontend dentro de web con `npm install`.
4. Generar o actualizar la base vectorial de apuntes con `python ingest.py`. Este paso carga los PDF, limpia texto, segmenta, genera embeddings y persiste los vectores en ChromaDB.
5. Levantar el sistema completo con `python app.py`. Este flujo inicia Docker, backend y frontend de forma coordinada. El backend queda usualmente en `http://127.0.0.1:8000` y la interfaz en `http://127.0.0.1:5173`.
6. Ejecutar la verificación automatizada con `python -m pytest testing`.
7. Revisar la evaluación funcional definida en `evaluation/functional_questions.json`, la cual se ejecuta como parte de las pruebas y valida las 35 preguntas del conjunto experimental.

Si se requiere ejecutar componentes por separado, el flujo manual equivalente es: `docker compose up -d --build` para PostgreSQL, pgAdmin y población inicial; `python -m uvicorn src.api.server:app --host 127.0.0.1 --port 8000` para el backend; y `npm run dev` dentro de web para el frontend. Esta ruta manual se usa principalmente para depuración. Para registrar trazas en Langfuse durante una ejecución manual, se debe habilitar `LANGFUSE_ENABLED=true` junto con las credenciales correspondientes, ya que el arranque automático de `app.py` desactiva Langfuse por defecto para facilitar la ejecución local.

XIX. CONCLUSIONES

El uso de una arquitectura multiagente permite que se puedan abordar distintos tipos de consultas mediante sus respectivos agentes, evitando depender de una única fuente de información o de un único flujo de procesamiento. La incorporación de un orquestador facilita la selección dinámica de herramientas y agentes, permitiendo adaptar el comportamiento del sistema según la intención del usuario.

Asimismo, la integración de recuperación documental, búsqueda web, memoria conversacional y acceso controlado a servicios externos proporciona una base bastante flexible para construir respuestas más contextualizadas. Este enfoque favorece la incorporación de nuevas capacidades sin modificar significativamente la estructura general de la solución.

Por otro lado, la delegación del acceso a la base de datos transaccional a través de un servidor MCP garantiza el cumplimiento del principio de mínimo privilegio. La aplicación de técnicas de enmascaramiento y anonimización en esta capa intermedia previene eficazmente la exposición de datos sensibles al modelo de lenguaje, consolidando un diseño completamente seguro y protegiendo la privacidad de la información.

Finalmente, la arquitectura propuesta demuestra la utilidad de combinar modelos de lenguaje con herramientas especializadas y mecanismos de trazabilidad, permitiendo construir sistemas más controlables, auditables y adecuados para escenarios en los cuales la procedencia de la información y la transparencia del proceso de obtención de respuesta son aspectos relevantes.

REFERENCIAS

- [1] M. Wooldridge and N. R. Jennings, "Intelligent agents: Theory and practice," *The Knowledge Engineering Review*, vol. 10, no. 2, pp. 115–152, 1995.
- [2] Y. Shoham and K. Leyton-Brown, *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press, 2008.
- [3] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, 2020.
- [4] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP*, 2019, pp. 3982–3992.
- [5] Google AI for Developers, "Gemini API Documentation," 2026. [Online]. Available: <https://ai.google.dev/api>
- [6] Tavily, "Tavily Search API Documentation," 2026. [Online]. Available: <https://docs.tavily.com/documentation/api-reference/endpoint/search>
- [7] Chroma, "Chroma Documentation," 2026. [Online]. Available: <https://docs.trychroma.com/>
- [8] Langfuse, "Langfuse Observability Documentation," 2026. [Online]. Available: <https://langfuse.com/docs/observability/overview>
- [9] FastAPI, "FastAPI Documentation," 2026. [Online]. Available: <https://fastapi.tiangolo.com/>